SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

QUESTIONS: 5 **Total Marks : 100**

Annexure A

Scenario based Questions

Code of Conduct:

I S.Mahalakshmi(192524372) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.MAHALAKSHMI

QUESTION 1 — CPU Scheduling in a Real-Time Ticket Booking System

1.1 Scenario Analysis

During festival seasons, a railway ticket booking system receives a large number of requests. The system must process payment transactions, check seat availability, and confirm and print tickets. Customers have reported that ticket confirmations are taking too long. The scheduling decision must therefore protect critical transactions while also ensuring that lower-priority work does not wait indefinitely.

Process	Function	Priority stated in scenario	Burst-time information
P1	Payment gateway transactions	High	Varies from 2 ms to 20 ms
P2	Seat availability checks	Medium	Varies from 2 ms to 20 ms
P3	Ticket confirmation and printing	Low	Varies from 2 ms to 20 ms

The main issue is the conflict between urgency and fairness. Payments must be handled reliably and quickly, while confirmation is also important because a customer needs to know whether a booking has succeeded. Giving P1 the highest priority is reasonable, but allowing P3 to wait forever would make the system unacceptable.

1.2 Selection of Scheduling Algorithm

The recommended algorithm is Preemptive Priority Scheduling with Aging. In priority scheduling, the CPU is assigned to the ready process with the highest priority. In the preemptive form, a newly arrived higher-priority process can interrupt a lower-priority

process. Aging is an additional fairness mechanism: the priority of a process increases gradually as its waiting time becomes longer.

This recommendation is a practical extension of the Priority Scheduling option listed in the question. It is not a claim that priority scheduling is always the best choice for every real-time system. Actual deadline guarantees require information about arrival times, execution times, deadlines, and resource constraints, which are not supplied here.

1.3 Working Principle

1. The OS places newly arrived processes in the ready queue.
2. The scheduler compares the priorities of ready processes.
3. The highest-priority ready process is selected to use the CPU.
4. If a higher-priority process arrives, it can preempt the running process.
5. Waiting time is monitored. Aging gradually increases the priority of processes that have waited too long.

1.4 Sample Execution Order and Gantt Chart

Assume for illustration that P1, P2, and P3 all arrive at time 0 and that no additional process arrives during the example. The initial order is P1 → P2 → P3 because their stated priorities are high, medium, and low.

Illustrative order: | P1 | P2 | P3 |

The problem states that burst times vary from 2 ms to 20 ms but does not give the exact burst time of each process. Therefore, exact start times, finish times, waiting times, and turnaround times cannot be calculated without making up data.

Process	**Initial Priority**	**Expected Scheduling Behaviour**
P1	High	Selected first when all three processes are ready.

P2	Medium	Runs after higher-priority work is completed or blocked.
P3	Low	Runs when higher-priority work permits; aging raises priority if it waits too long.

If a new high-priority payment process arrives while P3 is running, preemptive priority scheduling may interrupt P3. Once the high-priority work finishes or blocks, the scheduler can resume the waiting process.

1.5 How It Can Improve CPU Utilization and Waiting Time

- Urgent work receives CPU time quickly, reducing the waiting time of payment operations.
- When a process blocks for network or database I/O, the OS can schedule another ready process instead of waiting for the blocked process.
- Aging prevents low-priority confirmation tasks from being postponed indefinitely.
- A priority policy makes the system's response more predictable for critical operations.
- Correct lock management and database transaction handling remain necessary; CPU scheduling alone cannot fix a slow database or network bottleneck.

1.6 Drawbacks and Mitigation

The main drawback is starvation. If high-priority processes continuously arrive, low-priority ticket confirmation processes may wait for a very long time. Preemption can also introduce context-switching overhead. Furthermore, a priority scheduler by itself does not guarantee that every deadline will be met.

Mitigation: use aging, limit excessive priority levels, monitor waiting time, and measure end-to-end transaction latency. For a true real-time guarantee, the system designer should analyse task deadlines and worst-case execution times and select a real-time scheduling policy suitable for those requirements.

1.7 Conclusion

Preemptive Priority Scheduling with Aging is a reasonable choice for the ticket booking scenario because it gives urgent payment operations prompt access to the CPU while reducing starvation of ticket confirmation work. Its success depends on fair priority management, efficient I/O handling, and reliable transaction processing.

QUESTION 2 — Deadlock in a Banking Transaction Server

2.1 Scenario Analysis

The banking server handles three simultaneous operations: T1 transfers money between accounts, T2 updates passbook entries, and T3 generates monthly statements. The operations use three resources: A (Account Database), B (Customer Information Database), and C (Transaction Logs). The scenario states that T1 holds A and requests B, T2 holds B and requests C, and T3 holds C and requests A.

Transaction	**Resource currently held**	**Resource requested**	**Why it cannot proceed**
T1 — Money transfer	A	B	B is held by T2.
T2 — Passbook update	B	C	C is held by T3.
T3 — Monthly statement	C	A	A is held by T1.

2.2 Resource Allocation Graph (RAG)

A Resource Allocation Graph represents the relationship between processes and resources. Process nodes are shown in parentheses and resource nodes in square brackets. An arrow from a resource to a process means that the resource is allocated to that process. An arrow from a process to a resource means that the process is requesting the resource.

[A] ──allocated to──> (T1) ──requests──> [B]

[B] ──allocated to──> (T2) ──requests──> [C]

[C] ──allocated to──> (T3) ──requests──> [A]

Cycle in the graph: T1 → B → T2 → C → T3 → A → T1

This cycle shows that T1 is waiting for B, T2 is waiting for C, and T3 is waiting for A. Each requested resource is held by the next transaction in the cycle.

2.3 Deadlock Identification and Justification

Under the assumption that A, B, and C each have one relevant instance and that the locks cannot be forcibly taken away, a deadlock has occurred. All three transactions are waiting for resources that cannot be released while the transactions remain blocked.

Necessary condition	Meaning	Evidence in this scenario
Mutual exclusion	A resource can be used exclusively by one transaction at a time.	Each database or log lock is held by one transaction.
Hold and wait	A process holds at least one resource while requesting another.	T1 holds A and requests B; the same pattern applies to T2 and T3.
No preemption	A resource cannot simply be taken away from its holder.	The transactions wait for their current lock holders to release resources.

Circular wait	A cycle of processes waits for resources held by the next process.	T1 waits for T2, T2 waits for T3, and T3 waits for T1.

All four necessary conditions are present. Therefore, the cycle is not merely a possible risk under the stated single-instance assumptions; it represents a deadlock.

2.4 Recommended Technique: Prevention by Resource Ordering

The recommended prevention method is to impose a consistent global order on resources. For example, define A < B < C and require every transaction to acquire locks in ascending order. A transaction that needs A and B must obtain A before B. A transaction that needs A and C must obtain A before C.

When every transaction follows the same ordering rule, a transaction cannot hold a later resource and then request an earlier one. This breaks the circular-wait condition and prevents the same type of deadlock from forming.

2.5 Recovery if the Deadlock Has Already Occurred

Resource ordering prevents future deadlocks but does not automatically release locks in an existing deadlock. To recover, the database server can choose a transaction as a victim, roll it back to a safe state, release its locks, and retry it later. The victim should be selected carefully to minimise lost work and avoid repeatedly aborting the same transaction.

6. Detect the cycle using lock-wait information.
7. Select a transaction for rollback based on cost, progress, and impact.
8. Roll back safely and release its locks.
9. Allow the other transactions to proceed.
10. Retry the rolled-back transaction with a limit or back-off policy.

2.6 Locking Redesign and Banking Safety

- Document one global order for all database and log locks.
- Acquire required locks consistently and avoid acquiring extra locks that are not needed.
- Keep transactions short and release locks promptly.
- Use deadlock detection and lock timeouts as additional safeguards.
- Make money transfers atomic so that a partial transfer cannot leave account balances inconsistent.
- Make retry logic safe so a repeated request does not transfer the same amount twice.

2.7 Conclusion

The banking server is deadlocked because three transactions form a circular wait across resources A, B, and C. A consistent resource-ordering rule prevents this cycle from forming again, while deadlock detection and controlled rollback can recover from an existing deadlock. Transaction atomicity and safe retry logic are essential to preserve financial correctness.

QUESTION 3 — Mobile OS App Switching Lag

3.1 Scenario Analysis

The mobile operating system uses Round Robin scheduling with a time quantum of 20 ms and has 20 background applications active. Users report that opening WhatsApp delays background music and switching between social media applications feels laggy. The CPU also shows high idle time even though many applications are active.

The scenario has two distinct concerns: responsiveness for interactive applications and low CPU activity. The scheduling quantum may affect response time and overhead, but high idle time often indicates that processes are blocked or that no runnable task is available.

3.2 Why the CPU May Remain Idle

- I/O blocking: apps may be waiting for network replies, disk reads, file access, or other external operations.
- No runnable thread: an application can be active in the background but still have no thread ready to execute.
- Synchronization waits: a process may be waiting for a lock, event, or another process.
- I/O completion delay: if the OS does not promptly wake a process when I/O completes, the CPU may have less ready work.
- The number of active apps is not the same as the number of ready processes. Twenty background apps do not guarantee twenty runnable tasks.

Therefore, the 20 ms time quantum alone does not explain high CPU idle time. If no process is ready, changing the quantum will not create work for the CPU. The OS should examine ready-queue length, I/O wait time, thread states, and wake-up latency.

3.3 Effect of the 20 ms Quantum

Round Robin gives each ready process a time slice. When the slice expires, the process is placed at the back of the ready queue if it is still runnable. A longer quantum reduces how often the scheduler switches between processes, but it can make interactive tasks wait longer behind other runnable processes. A very short quantum improves opportunities to respond but increases context-switching overhead.

Quantum choice	Potential advantage	Potential disadvantage
Longer quantum	Fewer context switches and more continuous work per turn.	Interactive tasks may wait longer for their turn.
Shorter quantum	More frequent opportunities for waiting interactive tasks to run.	More context switches and scheduler overhead.
20 ms in this scenario	May reduce overhead compared with a much shorter slice.	May not be ideal for a workload that requires very fast interactive response.

The question does not provide measured context-switch cost or response-time targets, so it is not possible to prove from the given information that 20 ms is definitively too long.

3.4 Recommended Scheduling Method

Use preemptive priority scheduling for task classes, with Round Robin among processes of the same priority. Foreground interaction and audio-critical work can receive higher priority than non-urgent background synchronisation. A shorter

quantum, for example 5–10 ms, can be tested within a priority class, but the final value should be selected from measurements rather than assumed to be universally optimal.

Priority class	Example workload	Recommended treatment
High	Foreground interaction and audio-critical tasks	Prompt scheduling with careful latency monitoring.
Medium	Active application services	Regular CPU access and fair scheduling.
Low	Non-urgent background synchronisation	Run when possible, with aging or a guaranteed CPU share.

Aging or a minimum CPU share is needed to prevent low-priority work from being starved. Audio and other time-sensitive tasks also need suitable resource and I/O handling; priority alone does not guarantee smooth playback.

3.5 How the Changes Can Improve Performance

- Lower interactive waiting time: foreground tasks can be scheduled sooner.
- Smoother audio: audio-related work can receive CPU time before its playback buffer is exhausted.
- Reduced perceived lag: small interactive tasks need not wait behind long-running background tasks at the same priority.
- Controlled overhead: testing the quantum helps balance response time against context-switching cost.
- Better throughput: the scheduler can keep the CPU busy when runnable tasks exist, while efficient I/O handling can reduce avoidable delays.

A scheduling change cannot remove delays caused by a slow network, storage bottleneck, memory pressure, or a blocked dependency. Performance measurements

should include app-switch latency, audio dropouts, CPU idle percentage, ready-queue length, and context switches per second.

3.6 Conclusion

The scenario suggests that background I/O waits and a scheduling policy that does not sufficiently prioritise interactive tasks may contribute to poor responsiveness. Preemptive priority scheduling combined with Round Robin within priority classes is a reasonable design. The OS should test the time quantum and improve I/O wake-ups, while ensuring fairness to background applications.

QUESTION 4 — Deadlock Risk in an Operating System Update

4.1 Scenario Analysis

During a Windows OS update, several modules run in parallel: M1 copies update files, M2 installs drivers, M3 updates the registry, and M4 validates system integrity. The resources are R1 (Disk I/O), R2 (Kernel lock), and R3 (Configuration lock). The logs report that M1 holds R1 and waits for R2, M2 holds R2 and waits for R3, and M3 holds R3 and waits for R1.

Module	**Function**	**Resource held**	**Resource requested**
M1	Copy update files	R1 — Disk I/O	R2 — Kernel lock
M2	Install drivers	R2 — Kernel lock	R3 — Configuration lock
M3	Update registry	R3 — Configuration lock	R1 — Disk I/O
M4	Validate system integrity	Not specified	Not specified

4.2 Deadlock Identification

Yes. Assuming each resource has a single relevant instance and the locks are not forcibly preempted, M1, M2, and M3 are deadlocked. M4 is mentioned in the scenario, but no allocation or request information is supplied for M4; therefore, it cannot be included in the demonstrated cycle.

4.3 Exact Circular Wait

[R1] ──allocated to──> (M1) ──requests──> [R2]

[R2] ──allocated to──> (M2) ──requests──> [R3]

[R3] ──allocated to──> (M3) ──requests──> [R1]

Cycle: M1 → R2 → M2 → R3 → M3 → R1 → M1

11. M1 cannot proceed until it receives R2, which is held by M2.
12. M2 cannot proceed until it receives R3, which is held by M3.
13. M3 cannot proceed until it receives R1, which is held by M1.
14. Each module waits for the next module in the cycle, so none can release its held resource through normal completion.

4.4 Banker's Algorithm-Style Avoidance

Banker's algorithm avoids unsafe resource allocations by checking whether the system remains in a safe state after a request. A safe state is one in which there is some order of completing all processes so that each can obtain its remaining declared needs from the available resources plus resources released by completed processes.

Structure	**Meaning**
Available	Number of free instances of each resource type.
Allocation	Number of instances currently assigned to each module.
Max	Maximum number of instances each module may need.
Need	Remaining declared need: Max − Allocation.

Need = Max − Allocation

The algorithm follows these checks:

15. Confirm that a module's request does not exceed its declared Need.
16. Confirm that the requested resources are currently Available.

17. Temporarily allocate the requested resources and update Available, Allocation, and Need.
18. Run the safety test: repeatedly find a module whose remaining Need can be met by the current Work vector; assume it completes and releases its Allocation.
19. If every module can finish, the state is safe and the request may be granted. If not, restore the old state and make the module wait.

The assignment does not specify the number of instances of R1, R2, or R3, the Max matrix, or the full Allocation matrix. Therefore, a numerical Banker's algorithm safety sequence cannot be computed from the supplied information. The procedure above shows how an OS designer would apply the method once those values are available.

If the current state is already deadlocked, the OS must first recover by rolling back or terminating a selected module and releasing its resources. Banker's algorithm is an avoidance method, not a direct method for breaking an existing deadlock.

4.5 Prioritising Modules and Improving CPU Utilization

- Use dependency-aware scheduling so a module is not repeatedly scheduled when it cannot proceed because a required lock is unavailable.
- Prioritise critical update stages where appropriate, but do not ignore lock dependencies or integrity requirements.
- Use asynchronous disk I/O so the CPU can run other ready work while a module waits for disk completion.
- Keep critical sections short and release kernel and configuration locks promptly.
- Monitor lock wait times and detect cycles early.
- Provide fair access to resources so low-priority update modules do not starve.

4.6 Conclusion

The reported allocation pattern contains a circular wait among M1, M2, and M3, so it can cause deadlock. Banker's algorithm-style safety checking can prevent unsafe

allocations if the OS knows the maximum demands and resource counts. Dependency-aware scheduling, shorter lock holding times, asynchronous I/O, and deadlock recovery can improve update progress and resource utilisation.

QUESTION 5 — Cloud Server CPU Utilization Analysis

5.1 Scenario Analysis and Assumptions

A cloud hosting provider reports that virtual machine processes wait for network or disk 80% of the time and use the CPU 20% of the time. Each server runs seven VMs. The purpose of the calculation is to estimate the chance that the CPU is busy when multiple independent VMs can run.

Important source note: the formula is not legible in the supplied assignment text. The calculation below uses the standard multiprogramming model $U = 1 - p^n$, where p is the probability that one VM is waiting for I/O and n is the number of independent VMs. This formula and its assumptions should be confirmed against the faculty's original copy.

5.2 CPU Utilization Calculation

Given:

$p = 80\% = 0.8$

$n = 7$ VMs

Formula:

$U = 1 - p^n$

Substitute:

$U = 1 - (0.8)^7$

Calculate the power:

$(0.8)^7 = 0.2097152$

Therefore:

$U = 1 - 0.2097152 = 0.7902848$

CPU utilization $= 0.7902848 \times 100 \approx 79.03\%$

Estimated CPU Utilization = 79.03%

Interpretation: under the model, there is approximately a 79.03% chance that at least one of the seven VMs is not waiting for I/O and can use the CPU. This assumes independent I/O waiting, a CPU that can execute another ready VM, and no additional bottlenecks.

5.3 Underutilized or Optimally Loaded?

A utilization estimate of about 79.03% suggests that the CPU could be reasonably busy under the model. However, it does not by itself prove that the server is optimally loaded. Optimal loading depends on performance targets, response time, CPU saturation, memory pressure, and storage and network capacity.

- CPU metrics show how often the processor is busy, but not whether applications are meeting response-time targets.
- High disk latency can slow applications even when CPU utilization is moderate.
- Network congestion can cause delays that adding more VMs will not solve.
- Memory pressure or swapping can reduce performance and may increase I/O wait.

5.4 Effect of Changing the Number of VMs

Using the same assumptions and $U = 1 - (0.8)^n$, the estimates for different VM counts are:

Number of VMs (n)	**Calculation**	**Estimated CPU Utilization**
1	$1 - (0.8)^1$	20.00%
3	$1 - (0.8)^3$	48.80%
5	$1 - (0.8)^5$	67.23%

7	$1 - (0.8)^7$	79.03%
9	$1 - (0.8)^9$	86.58%

The estimates increase as the number of independent VMs increases, because it becomes more likely that at least one VM is ready to run while others wait for I/O. This is a simplified theoretical model, not a guarantee of real-world utilisation.

5.5 Increasing VM Count: Benefits and Risks

Potential benefits:

- More opportunities to execute a ready process while another VM waits for I/O.
- Potentially higher CPU utilisation and aggregate throughput.
- Better use of spare compute capacity.

Potential risks:

- Higher memory use and possible swapping.
- More competition for disk bandwidth, network bandwidth, and shared caches.
- Increased scheduling overhead and possible contention.
- Higher application latency if the host becomes overloaded.

5.6 Decreasing VM Count: Benefits and Risks

- Fewer VMs may reduce memory pressure and competition for shared storage or network resources.
- Individual VMs may receive more dedicated resources and achieve better response time.
- However, fewer independent workloads can mean fewer runnable tasks during I/O waits, reducing CPU utilisation and aggregate throughput.

Decision: increase VM density only while the host has adequate CPU, memory, storage, and network capacity. Monitor actual workload performance rather than trying to maximise CPU utilisation alone.

5.7 Two OS-Level Strategies to Improve CPU Utilization

Strategy 1: Effective Multiprogramming and Scheduling

The OS or hypervisor should schedule another ready process or VM when the current one blocks for I/O. Appropriate CPU shares, priorities, and fair scheduling help ensure that available CPU time is used while maintaining reasonable service for each VM. The scheduler should avoid assigning CPU time to a VM that has no runnable work.

Expected benefit: less idle CPU time when other runnable work exists, with potentially higher aggregate throughput.

Strategy 2: I/O Optimization

Use asynchronous I/O, buffering, suitable caching, and efficient I/O queue management. These techniques can reduce avoidable delays and allow a VM to continue other work while an I/O request is pending. Storage and network bottlenecks should also be measured and addressed at the appropriate layer.

Expected benefit: reduced avoidable waiting and better system throughput. These techniques cannot remove unavoidable network latency or slow physical storage, and they should not be treated as substitutes for sufficient hardware capacity.

5.8 Conclusion

Using the standard independent-I/O-wait model, CPU utilisation for seven VMs is estimated at 79.03%. Increasing the number of VMs can improve the chance that a process is ready to run, but excessive consolidation may create memory, disk, and network bottlenecks. Effective multiprogramming, fair CPU scheduling, and I/O optimisation can improve utilisation and throughput while preserving response time.

RUBRICS FOR CALCULATION- Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

TOTAL MARKS AWARDED **FACULTY SIGNATURE**